L M U ☀ C M S I 3 8 0 1

LANGUAGES AND AUTOMATA I

Syllabus • Fall, 2024

4 semester hours
Mondays and Wednesdays 1:45 p.m.–3:25 p.m. in Pereira 140
Instructor: Ray Toal, Doolan 110, rtoal@lmu.edu, +1 310.338.2773
Teaching Assistant(s): Natalie Lau and Diego Acosta
Slack channel (lmucs.slack.com): cmsi3801-fall-2024

Learning Outcomes

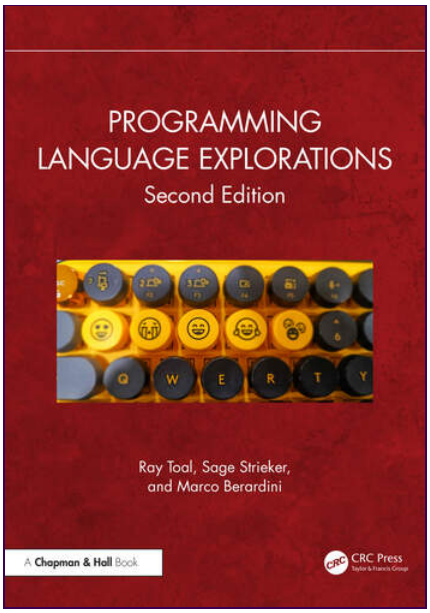
By the end of this course you will have:

- mastered fundamental concepts that underlie programming language syntax and semantics through a comparative study of several languages and their features;
- learned several new programming language features and paradigms;
- written code in several programming languages;
- gained the ability to study conceptual linguistic issues without being blinded by a particular language's implementation; and
- gained insight into the problem of designing new languages.

Prerequisites

Programming proficiency in one, but preferably two, high-level languages such as JavaScript, Python, Java, Ruby, C, C++, or C#. Previous courses in Data Structures and Algorithms are essential.

Readings



While no textbook is required for this course, the majority of the course material will come from:

- Ray Toal, Sage Strieker, and Marco Berardini, *Programming Language Explorations*, CRC Press, 2024.

You might also like another look at programming language concepts from:

- Robert W Sebesta, *Concepts of Programming Languages*, 12th Edition, Addison-Wesley, 2019.

Many students benefit a great deal from language-specific books. **However, keep in mind that programming languages evolve much more quickly than books are written about them.** Here are a few suggestions for your reading pleasure; just make sure you supplement your study with online sources:

- Apple, Inc., *The Swift Programming Language*.
- Axel Rauschmayer, *(Any or all of his JavaScript books)*.
- James Gosling, Bill Joy, Guy L. Steele, Jr., Gilad Bracha, Alex Buckley, and Daniel Smith, *The Java Language Specification, Java SE 22 Edition.*
- Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.
- Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd edition. Prentice Hall, 1988.
- *The Rust Programming Language* (online book).
- Miran Lipovača, Learn You a Haskell for Great Good, No Starch Press, 2011.

Various papers and readings will be assigned throughout the course (including my own course notes, practice problems, and sample code). These readings will be posted on

individual assignment pages. If you have projects or papers to work on, you'll have to find some additional readings on your own. Make sure you take the time for effective self-study. Take advantage of classmates and friends; the computing industry is one of the most collaborative fields in which to work, and your course experience should reflect this.

Workload

In accordance with the LMU Credit Hour Policy, this 4-unit course will require 12 hours of work per week (including the time spent in lecture and lab).

Assignments and Grading

You'll have several homework sets containing in-depth theoretical questions and non-trivial programming problems, and quizzes and a final exam with less difficult material. To help prepare you to meet industry expectations for college graduates, programming assignments will sometimes be required to be placed in version-controlled public repositories (most students tend to prefer GitHub). Exams may cover material from lectures *not previously assigned* for homework, so take care to learn concepts and not just memorize technical steps and recipes to answer questions.

Occasionally, you may be permitted to work in groups of students; however, while only one solution set is turned in per group, both students are responsible for understanding *all* of its content and may be asked at any time for an oral explanation of any solution. Collaboration with other groups is fine but must be limited: you may share ideas and approaches but not solutions. You **must** acknowledge any help received. Academic dishonesty may result in expulsion; be certain your work meets the standards set forth in the LMU Honor Code.

Your final grade will be weighted as follows:

Homework sets	50 pts
Exam 1	10 pts
Exam 2	15 pts

Final Exam ..... 25 pts

Letter grades are figured according to the usual scale: 90% or more of the total points guarantees you an **A**, 80% a **B**, 70% a **C**, and so on. Note the word *guarantee*: 82 points will earn you *at least* a **B-**; you might still get an **A** if 82 is or is near the top score. The lower bounds ensure grades measure your achievement of the learning outcomes, and can never punish you because you did very well but on the low end of a class full of high-achievers.



To ensure a degree of fairness for those who consistently perform timely course work, and to encourage everyone to pace themselves properly in completing assignments, late work is normally penalized 15% per 24-hour period. If there is an issue that prevents you from submitting an assignment on time (e.g., excellent surf conditions, personal or family issues, sickness, conference attendance, job interviews, a family ski trip, or personal emergencies), let the instructor know ahead of time.

Where assignments involve programming, the *quality* of your code, not just its *correctness*, will play a large part in determining your grade. Please refer to these resources and notes on clean code for information on expectations of code quality. Appearance of the grading policy in this syllabus constitutes fair warning of the consequences of poorly written code.

If the course has a contribution score, it will be computed by awarding you 1 point every time you: (1) correct me during class, (2) ask a profound question during class (where profound elevates the discussion in some sense), (3) answer a question during class as the first response, (4) post a good question on the class Slack channel, (5) post a detailed answer to another student's question on the Slack channel, (6) recommend an article or video or tutorial (on the Slack channel) directly related to the course material. There is an expectation of 10 points throughout the semester, well-spaced. (Please do not dump 10 video recommendations on Slack during the first week of class and then shut down.)

Important Dates

When	What
2024-09-02 (Monday)	No class (Labor Day)
2024-09-13 (Friday)	Homework 1 Due
2024-10-04 (Friday)	Homework 2 Due
2024-10-07 (Monday)	Exam 1
2024-10-10 (Thursday)	Midterm deficiency grades due
2024-11-01 (Friday)	Homework 3 Due
2024-11-13 (Wednesday)	Exam 2
2024-11-15 (Friday)	Last day to drop
2024-11-22 (Friday)	Homework 4 Due
2024-11-27 (Wednesday)	No class (Thanksgiving Break Wednesday)
2024-12-06 (Friday)	Homework 5 Due
2024-12-09 (Monday)	Final Exam

Student Rights and Responsibilities

You have the right to:

- A syllabus with stated learning outcomes
- Have those learning outcomes addressed
- Chats with the instructor during office hours
- Have questions answered via email (one-day turnaround max)
- Video and text chats with the instructor
- Feedback on your work
- A challenging experience (you should be working a little outside your comfort zone)
- All lecture sessions recorded and stored online for you to access
- An experience beyond what you would get from books, websites, online tutorials, and discussion forums
- Feel like you belong (if you are experiencing impostor syndrome or stereotype threat, please see the instructor)

- Leave the course better skilled than when you came in

In return, you are expected to:

- Use the lab resources, TA time, and instructor time when you need them
- Stay on top of all course announcements, both on BrightSpace and Slack
- Follow all homework submission instructions exactly
- Watch the course recording if you miss class
- Perform work conscientiously, neatly, and in a timely manner
- Be considerate and respectful to your fellow students
- Be honest with yourself and others
- Finish the course or withdraw by the withdrawal deadline

For online courses, I recommend your camera be turned on unless you have bandwidth problems or need to step away.

Two Tips for Success

You've probably seen hundreds of these, but there are two that stand out.

- ✅ **Be aware of how students trick themselves into thinking they're learning.** Cramming, rereading, and highlighting do **not** move information from your working memory into long-term memory. Short-term working memory is not "learned." Recall is the most effective way to load your long-term memory. Please work through Nicky Case's interactive essay How to remember everything forever-ish. (Also, this article backs up the importance of recall.)
- ✅ **Use checklists**, as these are incredibly effective. Read Atul Gawande's The Checklist and think about how you apply these to your studies.

Generative AI and Your Learning

Academic honesty includes the appropriate use of technology as an aid for learning and productivity. This includes but is not limited to LLM-based generative artificial

intelligence tools such as ChatGPT, Gemini, and Copilot.

Modern GenAI provides fast solutions to a variety of computing problems but must be used responsibly to get the most out of your education. GenAI will not always be helpful, especially when asked to write code for scenarios it cannot recognize. It cannot independently validate the code that it produces, and will frequently produce bugs and security vulnerabilities, making things worse for you. It will not be available for job interviews. As a student and future software professional, you have a moral and ethical responsibility to deeply understand every line of the software you author and should therefore employ GenAI in your coursework **sparingly**, especially when its use might rob you of the stretch-zone learning that comes from crafting programs on your own.

That said, there are several acceptable use cases of GenAI in programmatic assignments, including:

- Generating short documentation strings and type hints.
- Finding examples and explanations for syntactic forms (e.g., how to use the spread operator; how to assign variables directly from an array or object; how to construct a dictionary comprehension; how to unwrap an optional, and so on). Prompt the AI for the how-to's, *not* your final code.
- Interpreting error messages, as these might be pretty cryptic! Sometimes the AI can be a useful tutor when explaining things it is familiar with.
- Generating unit tests for edge cases. When it finds something you overlooked, make a mental or written note so you can find these cases on your own next time.
- Brainstorming new ideas for apps.

> ***Remember the four As***
>
> *DO use technologies such as GenAI to **augment** ✅, **amplify** ✅, and **accelerate** ✅ your learning; NEVER use technology to **avoid** ❌ learning.*

> ***Be responsible!***
>
> *Generally, using GenAI and related tools without knowing what you are doing often leads to poor homework submissions that rarely achieve a grade higher than*

an F on their own merits, simply due to hallucinations and an inability to carry out tasks as they were assigned (even with good prompts).

Also, instructors and TAs are GenAI users themselves and will be able to easily detect inappropriate student use of these tools. The teaching staff will try to steer you back on track should we feel like you are giving up on your learning.

If you would like to use GenAI appropriately as a learner, see this short introductory article. If you are a TA, see this article by Jeff Olson on how to help teach students to use GenAI.

Topics

We'll cover most of the following topics in the order listed. Keep in mind this is only the first half of a year-long programming languages sequence, so topics you don't see here are likely to appear in the second course.

- UNIT 0—THE STUDY OF PROGRAMMING LANGUAGES: Why and how to study programming languages; A brief history; A list of influential languages with code samples; Introduction to language concepts and formalisms; Pragmatic concerns; Evaluation criteria; Classification; Programming paradigms.
- UNIT 1—BASICS AND SCRIPTING: Names and entities; Declarations, expressions, and statements; Binding, scope (static and dynamic) and extent (static, stack, and heap); Constants, variables, types, functions, and modules; Assignments, function calls, and pattern matching; Higher order functions, closures and shallow vs. deep binding; Aliasing, overloading, and polymorphism. Operator precedence, associativity, arity and fixity; Evaluation order, short-circuiting; Structured and unstructured control flow: sequencing, selection, iteration, recursion, and exceptions; Coroutines, promises, and async-await. **Covered Languages: Lua, Python, JavaScript**.
- UNIT 2—ENTERPRISE LANGUAGES: Abstraction, Modules, objects, encapsulation, information hiding; Tenets of object-orientation: encapsulation, inheritance and dynamic method binding; Issues with multiple inheritance; Implementation issues;

Class-based vs. prototype-based systems; Pure vs. hybrid object systems; Streams; Abstract and generic types; Covariance, Contravariance, and Invariance. **Covered Languages: Java, Kotlin, Swift**.

- UNIT 3—TYPES: Type systems; Static vs. dynamic typing, strong vs. weak typing, manifest vs. implicit typing, and nominal vs. structural typing; Type checking, type equivalence, type coercion, and type inference; Primitive types: numbers, characters, and symbols; Sum types and product types; Unions and subtypes; Collections: arrays, sets, streams, regular expressions; Dependent types; Introduction to Type Theory. **Covered Languages: Standard ML, OCaml, F#, Haskell, TypeScript**.

- UNIT 4—SYSTEMS LANGUAGES: The runtime stack and activation records; Internal representation of values; Calling conventions; Passing arguments and returning values; Pointers; Manual memory management; Smart pointers; How OOP is implemented behind the scenes: Ownership, borrowing, and lifetimes; The operating system interface. **Covered Languages: C, C++, Rust, Zig**.

- UNIT 5—CONCURRENCY: Motivation; Threads vs. events; Asynchronous programming: callbacks, promises, and async/await; Multiprocessing vs. multithreading; Communication and synchronization issues; Shared memory vs. message passing; Language-intrinsic concurrency vs. library managed concurrency; Implementation. **Covered Languages: Go, Erlang, Elixir, Gleam, Mojo**.

- UNIT 6—NON-TRADITIONAL LANGUAGES AND PARADIGMS: Logic programming; Constraint programming; Dataflow programming; Array programming; Concatenative programming; Esoteric programming languages. Examples will be taken from Prolog, APL, K, Forth, Factor, Brainf**k, Pyth, CJam, and others.

Additional Information

Tentative Nature of the Syllabus. If necessary, this syllabus and its contents are subject to revision; students are responsible for any changes or modifications announced or distributed in class or posted online.

Course Evaluation. Student feedback on this course provides valuable information for continued improvement. All students are expected to fairly and thoughtfully complete a



course evaluation for this course. Course evaluations for the Seaver College of Science and Engineering are administered online through the Blue™ evaluation system. You will receive an e-mail notification at your Lion e-mail address when the evaluation form is available. You may also access the evaluation form on Brightspace during the evaluation period. A few minutes of class time will be reserved for you to complete a course evaluation near the end of the semester. Please bring a laptop, smart phone, tablet or other mobile device to class on this date so that you can access the online evaluation platform.

Academic Honesty. Loyola Marymount University is a community dedicated to academic excellence, student- centered education and the Jesuit and Marymount traditions. As such, the University expects all members of its community to act with honesty and integrity at all times, especially in their academic work. Academic honesty respects the intellectual and creative work of others, flows from dedication and pride in performing one's own best work, and is essential if true learning is to take place. As an LMU Lion, you are pledged to join the discourse of the academy with honesty of voice and integrity of scholarship.

Academic dishonesty will be treated as an extremely serious matter with severe consequences that can range from receiving no credit for an assignment or test to failing the class, to expulsion. It is never permissible to turn in any work not been authored by you without properly acknowledging the source. It is your responsibility to make sure that your work meets the standard of academic honesty set forth in the "LMU Honor Code and Process" which appears in the LMU Bulletin.

Special Accommodations. Students with special needs who require reasonable modifications, special assistance, or accommodations in this course should promptly direct their request to the Disability Support Services (DSS) Office. Any student who currently has a documented disability (ADHD, Autism Spectrum Disorder, Learning, Physical, or Psychiatric) needing academic accommodations should contact the DSS Office in Daum Hall, as early in the semester as possible. All discussions will remain confidential. Please visit LMU DSS for additional information. Please request any needed assistance as early in the semester as possible.

Wellness. Please familiarize yourself with, and make use, of the resources at Lion Wellness as needed.

Reporting Requirements of Sexual or Interpersonal Misconduct. As responsible employees, faculty are required to report any case of suspected sexual or interpersonal misconduct and cannot protect student confidentiality. For information about confidential counseling on campus and for general information about consensual relationships, sexual harassment, and sexual assault, please visit LMU Cares.

Emergency Preparedness. To report an emergency or suspicious activity, phone the LMU Department of Public Safety (x222 or 310-338-2893) or at the nearest emergency call box. In the event of an evacuation, follow the evacuation signage throughout the building to the designated safe refuge area where you will receive further instruction from Public Safety or a Building Captain. For more safety information and preparedness tips, visit LMU DPS.

Ray Toal • *Computer Science Department* • *College of Science and Engineering* • *Loyola Marymount University*